MINI-PROJECT REPORT

DAYTONE - AI MENTAL WELLNESS TRACKER AND BURNOUT PREDICTION

Submitted By:

S A SANUSH

REGISTER NO: _____

DEPARTMENT OF COMPUTER APPLICATIONS

LOURDES MATHA COLLEGE OF SCIENCE AND TECHNOLOGY

KUTTICHAL, THIRUVANANTHAPURAM - 695574

(AFFILIATED TO APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY, KERALA (KTU))

(MANAGED BY ARCHDIOCESE OF CHANGANASSERY)

DAYTONE - AI MENTAL WELLNESS TRACKER AND BURNOUT PREDICTION

A Project Report

Submitted By,

S A SANUSH - _____

in partial fulfilment of the requirements for the award of
the degree in

MASTER OF COMPUTER APPLICATIONS

at

DEPARTMENT OF COMPUTER APPLICATIONS
LOURDES MATHA COLLEGE OF SCIENCE AND TECHNOLOGY
KUTTICHAL, THIRUVANANTHAPURAM - 695574
(AFFILIATED TO APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY, KERALA)

OCTOBER 2025

LOURDES MATHA COLLEGE OF SCIENCE AND TECHNOLOGY
KUTTICHAL, THIRUVANANTHAPURAM - 695574
(Affiliated to APJ Abdul Kalam Technological University, Kerala)
DEPARTMENT OF COMPUTER APPLICATIONS

CERTIFICATE

This is to certify that the project work entitled **"DAYTONE - AI MENTAL WELLNESS TRACKER AND BURNOUT PREDICTION"** is a Bonafide record of the work done by **S A SANUSH**, Reg No: _____, student of Department of Computer Applications, Lourdes Matha College of Science and Technology, Kuttichal, Thiruvananthapuram, affiliated to the APJ Abdul Kalam Technological University, Kerala from July 2025 to October 2025 in partial fulfilment of the requirements for the award of the Degree of Master of Computer Applications from APJ Abdul Kalam Technological University, Kerala.

Ms. ASHA CHANDRAN S Date:

(Internal Guide)

Ms. BISMI K CHARLEYS

(Head of the Department)

DECLARATION

I undersigned hereby declare that the project report **"DAYTONE - AI MENTAL WELLNESS TRACKER AND BURNOUT PREDICTION"** submitted for partial fulfilment of the requirements for the award of degree of Master of Computer Applications of the APJ Abdul Kalam Technological University, Kerala. This submission represents my idea in my own words and I have adequately and accurately cited and referenced the original sources.

I also declare that I have adhered to ethics of academic honesty and integrity and have not misrepresented or fabricated any data or idea or fact of source in my submission. I understand that any violation of the above will be a cause for disciplinary action by the institute and/or the university.

Place: TRIVANDRUM S A SANUSH
Date:

ACKNOWLEDGEMENT

An endeavour over a long time can be successful only with the advice, encouragement, and support of many well-wishers. I place on record my sincere gratitude to all those who directly or indirectly contributed to the successful completion of this project work.

At the outset, I thank God Almighty for the strength, guidance, and blessings received throughout the period of this Master of Computer Applications project.

It is a great pleasure to express my sincere gratitude to the Director and Principal of Lourdes Matha College of Science and Technology for permitting me to do this project with full support and encouragement.

I am highly obliged to Ms. BISMI K CHARLEYS, Head of the Department of Computer Applications of Lourdes Matha College of Science and Technology, for being a source of inspiration throughout the course and for valuable guidance.

With heartfelt thanks, I take this opportunity to thank my internal guide Ms. ASHA CHANDRAN S, Assistant Professor, and all staff members of the Department of Computer Applications for their continuous support, encouragement, and suggestions in various stages of the development of this project.

I also express my sincere gratitude to open-source communities whose tools and documentation helped in completing the DayTone project. Finally, I thank my friends and family who directly or indirectly contributed to this venture.

S A SANUSH

CONTENTS

TABLE OF CONTENT

ABSTRACT 1

CHAPTER 1 INTRODUCTION 2

1.1 GENERAL INTRODUCTION 3

1.2 GOAL OF THE PROJECT 4

CHAPTER 2 LITERATURE SURVEY 5

2.1 EXISTING SYSTEM 6

2.2 DRAWBACK OF EXISTING SYSTEM 6

CHAPTER 3 OVERALL DESCRIPTION 8

3.1 PROPOSED SYSTEM 9

3.2 FEATURES OF PROPOSED SYSTEM 9

3.3 REQUIREMENT SPECIFICATION 10

3.4 FEASIBILITY ANALYSIS 10

3.4.1 TECHNICAL FEASIBILITY 11

3.4.2 OPERATIONAL FEASIBILITY 11

3.4.3 ECONOMICAL FEASIBILITY 12

3.4.4 BEHAVIORAL FEASIBILITY 12

CHAPTER 4 OPERATING ENVIRONMENT 13

4.1 REQUIREMENTS 14

4.1.1 HARDWARE REQUIREMENTS 14

4.1.2 SOFTWARE REQUIREMENTS 14

CHAPTER 5 DESIGN 15

5.1 SYSTEM DESIGN 16

5.2 DATA FLOW DIAGRAM 18

5.3 INPUT DESIGN 20

5.4 OUTPUT DESIGN 22

CHAPTER 6 FUNCTIONAL &NON- FUNCTIONAL REQUIREMENTS 24

CONTENTS

6.1 FUNCTIONAL REQUIREMENTS 25

6.2 NON - FUNCTIONAL REQUIREMENTS 26

CHAPTER 7 ALGORITHM 28

7.1 DECISION TREE ALGORITHM (ML MODEL) 29

CHAPTER 8 TESTING 30

8.1 SYSTEM TESTING 31

8.2 UNIT TESTING 31

8.3 INTEGRATION TESTING 32

8.4 BLACK BOX TESTING 33

8.5 VALIDATION TESTING 33

8.6 OUTPUT TESTING 34

8.7 USER ACCEPTANCE TESTING 34

8.8 WHITE BOX TESTING 35

CHAPTER 9 APPENDICES 36

9.1 SCREENSHOTS 37

CHAPTER 10 CONCLUSION 45

10.1 CONCLUSION 46

10.2 FUTURE ENHANCEMENT 47

REFERENCES 48

ABSTRACT

DayTone is an AI-assisted mental wellness tracking web application designed to help users record daily mood, sleep, stress, activity, social interaction, and journal reflections. The system applies local Natural Language Processing and Machine Learning to detect emotional patterns, classify burnout risk, and provide meaningful wellness insights while preserving user privacy. DayTone is designed with emphasis on privacy, usability, maintainability, and academic clarity. The implementation uses modular Flask blueprints, SQLAlchemy models, local NLP processing, and Scikit-learn prediction so that the system can be explained, tested, and improved in a structured way. The design also considers practical deployment requirements such as migrations, secure configuration, logging, data export, and administrator monitoring.

The application is developed using Python Flask for the backend, SQLAlchemy for database handling, Scikit-learn for burnout prediction, NLTK VADER for sentiment analysis, Bootstrap and Jinja templates for the interface, and Chart.js, D3.js, and Three.js for interactive visualizations. DayTone also includes secure authentication, encrypted journal storage, PDF/CSV export, admin analytics, reminder emails, audit logs, model monitoring, and Progressive Web App support.

By combining daily self-reporting, local text analysis, burnout classification, and visual dashboards, DayTone provides a practical and privacy-conscious platform for early self-awareness and proactive wellness management. The project is intended as an academic wellness-support system and not as a clinical diagnostic tool.

In DayTone, Abstract is treated as a practical part of the complete wellness tracking workflow. The project does not present this topic only as a theoretical concept it connects the topic with actual screens, database records, backend routes, model outputs, and user decisions. This makes the report useful for explaining how the application works internally and how each module contributes to the final user experience.

The implementation follows a privacy-aware design approach. User wellness data may include personal habits, emotional reflections, stress values, and sleep patterns, so every major module is planned with confidentiality and controlled access in mind. Authentication, role-based pages, optional encryption, CSRF protection, and audit records support this requirement throughout the application.

From a software engineering perspective, the topic is implemented using modular Flask blueprints and reusable helper utilities. This makes the project easier to test and maintain because authentication, mood logging, administration, prediction, reports, mail, suggestions, and analytics are kept in separate areas of the codebase. A modular structure also makes future enhancement possible without rewriting the entire system.

From the user perspective, the topic is important because DayTone must remain simple and understandable. The system converts technical analysis into visible outputs such as dashboard cards, charts, heatmaps, risk labels, suggestions, history tables, and downloadable reports. The user is not expected to understand machine learning internals in order to benefit from the result.

CHAPTER 1
INTRODUCTION

1.1 GENERAL INTRODUCTION

Mental wellness has become an important concern among students and professionals because stress, lack of sleep, academic pressure, social isolation, and continuous screen time can gradually lead to burnout. Many people notice these patterns only after the problem becomes severe. A digital wellness tracker can help users observe daily habits, identify negative trends, and take preventive action. DayTone is designed with emphasis on privacy, usability, maintainability, and academic clarity. The implementation uses modular Flask blueprints, SQLAlchemy models, local NLP processing, and Scikit-learn prediction so that the system can be explained, tested, and improved in a structured way. The design also considers practical deployment requirements such as migrations, secure configuration, logging, data export, and administrator monitoring.

DayTone records day-to-day wellness data and converts it into understandable insights. Instead of only storing notes, the system evaluates numerical check-in values and journal text to generate sentiment scores, burnout risk levels, dashboard charts, history records, and personal suggestions.

The system focuses on privacy-first implementation. Journal notes can be encrypted at rest, sentiment analysis is performed locally, and users can export or delete their records. These design choices make the project suitable for academic demonstration of ethical software design.

In DayTone, 1.1 General Introduction is treated as a practical part of the complete wellness tracking workflow. The project does not present this topic only as a theoretical concept it connects the topic with actual screens, database records, backend routes, model outputs, and user decisions. This makes the report useful for explaining how the application works internally and how each module contributes to the final user experience.

The implementation follows a privacy-aware design approach. User wellness data may include personal habits, emotional reflections, stress values, and sleep patterns, so every major module is planned with confidentiality and controlled access in mind. Authentication, role-based pages, optional encryption, CSRF protection, and audit records support this requirement throughout the application.

From a software engineering perspective, the topic is implemented using modular Flask blueprints and reusable helper utilities. This makes the project easier to test and maintain because authentication, mood logging, administration, prediction, reports, mail, suggestions, and analytics are kept in separate areas of the codebase. A modular structure also makes future enhancement possible without rewriting the entire system.

From the user perspective, the topic is important because DayTone must remain simple and understandable. The system converts technical analysis into visible outputs such as dashboard cards, charts, heatmaps, risk labels, suggestions, history tables, and downloadable reports. The user is not expected to understand machine learning internals in order to benefit from the result.

1.2 GOAL OF THE PROJECT

The goal of DayTone is to provide a simple, useful, and secure wellness monitoring system that encourages daily self-reflection and early burnout awareness. The system does not replace professional care, but it helps users recognize repeated negative patterns in sleep, stress, activity, mood, and personal reflections. DayTone is designed with emphasis on privacy, usability, maintainability, and academic clarity. The implementation uses modular Flask blueprints, SQLAlchemy models, local NLP processing, and Scikit-learn prediction so that the system can be explained, tested, and improved in a structured way. The design also considers practical deployment requirements such as migrations, secure configuration, logging, data export, and administrator monitoring.

The project aims to combine a friendly interface with intelligent analysis. The user should be able to enter a daily log, immediately view the effect of the log in dashboard charts, and understand whether the pattern suggests Low, Medium, or High burnout risk.

Another goal is to show how academic machine learning systems can be implemented responsibly. The report clearly identifies the limits of synthetic training data and treats DayTone as a wellness-awareness platform rather than a clinical decision system.

In DayTone, 1.2 Goal Of The Project is treated as a practical part of the complete wellness tracking workflow. The project does not present this topic only as a theoretical concept it connects the topic with actual screens, database records, backend routes, model outputs, and user decisions. This makes the report useful for explaining how the application works internally and how each module contributes to the final user experience.

The implementation follows a privacy-aware design approach. User wellness data may include personal habits, emotional reflections, stress values, and sleep patterns, so every major module is planned with confidentiality and controlled access in mind. Authentication, role-based pages, optional encryption, CSRF protection, and audit records support this requirement throughout the application.

From a software engineering perspective, the topic is implemented using modular Flask blueprints and reusable helper utilities. This makes the project easier to test and maintain because authentication, mood logging, administration, prediction, reports, mail, suggestions, and analytics are kept in separate areas of the codebase. A modular structure also makes future enhancement possible without rewriting the entire system.

From the user perspective, the topic is important because DayTone must remain simple and understandable. The system converts technical analysis into visible outputs such as dashboard cards, charts, heatmaps, risk labels, suggestions, history tables, and downloadable reports. The user is not expected to understand machine learning internals in order to benefit from the result.

In DayTone, 1.2 Goal Of The Project is treated as a practical part of the complete wellness tracking workflow. The project does not present this topic only as a theoretical concept it connects the topic with actual screens, database records, backend routes, model outputs, and user decisions. This makes the report useful for explaining how the application works internally and how each module contributes to the final user experience.

CHAPTER 2
LITERATURE SURVEY

2.1 EXISTING SYSTEM

The existing method for wellness monitoring is usually manual logging in notebooks, spreadsheets, or basic mobile apps. These systems store user inputs but provide only simple summaries. In many cases, the user must manually compare dates and interpret patterns without support from an intelligent model. DayTone is designed with emphasis on privacy, usability, maintainability, and academic clarity. The implementation uses modular Flask blueprints, SQLAlchemy models, local NLP processing, and Scikit-learn prediction so that the system can be explained, tested, and improved in a structured way. The design also considers practical deployment requirements such as migrations, secure configuration, logging, data export, and administrator monitoring.

Some commercial platforms provide analytics, but they often depend on proprietary cloud processing. This can create privacy concerns when the user records sensitive personal reflections. They may also lack transparent explanation of how risk or recommendation scores are generated.

2.2 DRAWBACK OF EXISTING SYSTEM

Although existing systems support basic logging, they are not sufficient for a privacy-aware intelligent wellness platform. They either lack predictive power, lack security controls, or lack meaningful visual analytics. DayTone is designed with emphasis on privacy, usability, maintainability, and academic clarity. The implementation uses modular Flask blueprints, SQLAlchemy models, local NLP processing, and Scikit-learn prediction so that the system can be explained, tested, and improved in a structured way. The design also considers practical deployment requirements such as migrations, secure configuration, logging, data export, and administrator monitoring.

Most diary applications store entries but do not classify burnout risk. Habit trackers may show streaks but do not interpret emotional state. Cloud-based AI tools may process sensitive data outside the user environment. These gaps reduce user trust and limit practical usefulness.

2.2 DRAWBACK OF EXISTING SYSTEM (CONTINUED)

This continuation page expands the previous discussion with additional implementation details, academic observations, and practical DayTone examples.

From a software engineering perspective, the topic is implemented using modular Flask blueprints and reusable helper utilities. This makes the project easier to test and maintain because authentication, mood logging, administration, prediction, reports, mail, suggestions, and analytics are kept in separate areas of the codebase. A modular structure also makes future enhancement possible without rewriting the entire system.

From the user perspective, the topic is important because DayTone must remain simple and understandable. The system converts technical analysis into visible outputs such as dashboard cards, charts, heatmaps, risk labels, suggestions, history tables, and downloadable reports. The user is not expected to understand machine learning internals in order to benefit from the result.

In DayTone, 2.2 Drawback Of Existing System is treated as a practical part of the complete wellness tracking workflow. The project does not present this topic only as a theoretical concept it connects the topic with actual screens, database records, backend routes, model outputs, and user decisions. This makes the report useful for explaining how the application works internally and how each module contributes to the final user experience.

The implementation follows a privacy-aware design approach. User wellness data may include personal habits, emotional reflections, stress values, and sleep patterns, so every major module is planned with confidentiality and controlled access in mind. Authentication, role-based pages, optional encryption, CSRF protection, and audit records support this requirement throughout the application.

From a software engineering perspective, the topic is implemented using modular Flask blueprints and reusable helper utilities. This makes the project easier to test and maintain because authentication, mood logging, administration, prediction, reports, mail, suggestions, and analytics are kept in separate areas of the codebase. A modular structure also makes future enhancement possible without rewriting the entire system.

From the user perspective, the topic is important because DayTone must remain simple and understandable. The system converts technical analysis into visible outputs such as dashboard cards, charts, heatmaps, risk labels, suggestions, history tables, and downloadable reports. The user is not expected to understand machine learning internals in order to benefit from the result.

From the academic perspective, the topic demonstrates how web development, database management, data visualization, natural language processing, machine learning, security, and documentation can be combined into a single MCA project. It also shows awareness of limitations, especially the fact that the burnout model is trained on synthetic and semi-real validation data and is not a clinical diagnostic system.

CHAPTER 3
OVERALL DESCRIPTION

3.1 PROPOSED SYSTEM

The proposed system, DayTone, is a Flask-based mental wellness tracker that allows registered users to submit daily logs and view personalized analytics. The backend computes sentiment from journal notes, extracts behavioural features, predicts burnout risk, stores history, and generates suggestions. DayTone is designed with emphasis on privacy, usability, maintainability, and academic clarity. The implementation uses modular Flask blueprints, SQLAlchemy models, local NLP processing, and Scikit-learn prediction so that the system can be explained, tested, and improved in a structured way. The design also considers practical deployment requirements such as migrations, secure configuration, logging, data export, and administrator monitoring.

The frontend displays charts, heatmaps, risk status, goals, history, and a WebGL mood orb. Administrators can monitor users, audit logs, invite tokens, model feedback, and platform analytics. The system therefore combines user self-care features with administrative tools.

3.2 FEATURES OF PROPOSED SYSTEM

DayTone combines daily logging, analysis, visualization, security, and export features in one application. The system is modular, so each function is handled by a separate blueprint or utility module. DayTone is designed with emphasis on privacy, usability, maintainability, and academic clarity. The implementation uses modular Flask blueprints, SQLAlchemy models, local NLP processing, and Scikit-learn prediction so that the system can be explained, tested, and improved in a structured way. The design also considers practical deployment requirements such as migrations, secure configuration, logging, data export, and administrator monitoring.

Important user features include registration, login, profile settings, daily mood log, dashboard charts, history page, goals, heatmap, PDF report export, and CSV export. These features make the application useful for regular self-monitoring.

3.3 REQUIREMENT SPECIFICATION

The requirement specification defines the technologies, data, security controls, and user operations needed for DayTone. The system must be easy enough for daily use and strong enough to protect personal wellness records. DayTone is designed with emphasis on privacy, usability, maintainability, and academic clarity. The implementation uses modular Flask blueprints, SQLAlchemy models, local NLP processing, and Scikit-learn prediction so that the system can be explained, tested, and improved in a structured way. The design also considers practical deployment requirements such as migrations, secure configuration, logging, data export, and administrator monitoring.

Frontend requirements include HTML, CSS, Bootstrap, JavaScript, Chart.js, D3.js, and Three.js. Backend requirements include Python Flask, Flask-Login, Flask-WTF, Flask-Migrate, Flask-Limiter, Flask-Mail, SQLAlchemy, Scikit-learn, NLTK VADER, and ReportLab.

3.4 FEASIBILITY ANALYSIS

Feasibility analysis studies whether the proposed system can be built, operated, maintained, and accepted by users. DayTone is feasible because it uses open-source tools, a simple interface, lightweight processing, and a modular design. DayTone is designed with emphasis on privacy, usability, maintainability, and academic clarity. The implementation uses modular Flask blueprints, SQLAlchemy models, local NLP processing, and Scikit-learn prediction so that the system can be explained, tested, and improved in a structured way. The design also considers practical deployment requirements such as migrations, secure configuration, logging, data export, and administrator monitoring.

The project can be developed and tested on a normal laptop. It can later be deployed using a web service, database, and Redis-backed rate limiting. This makes it suitable for a student project and for future improvement.

3.4.1 TECHNICAL FEASIBILITY

3.5.1 Technical Feasibility is an important part of the DayTone project report. This section explains how the selected concept supports the overall objective of creating a secure, privacy-aware, AI-assisted mental wellness tracker. DayTone is designed with emphasis on privacy, usability, maintainability, and academic clarity. The implementation uses modular Flask blueprints, SQLAlchemy models, local NLP processing, and Scikit-learn prediction so that the system can be explained, tested, and improved in a structured way. The design also considers practical deployment requirements such as migrations, secure configuration, logging, data export, and administrator monitoring.

In this part of the project, the design decisions are connected to practical implementation. The section describes the role of the module, its interaction with other modules, the data it uses, and the expected result produced for users or administrators.

3.4.2 OPERATIONAL FEASIBILITY

3.5.2 Operational Feasibility is an important part of the DayTone project report. This section explains how the selected concept supports the overall objective of creating a secure, privacy-aware, AI-assisted mental wellness tracker. DayTone is designed with emphasis on privacy, usability, maintainability, and academic clarity. The implementation uses modular Flask blueprints, SQLAlchemy models, local NLP processing, and Scikit-learn prediction so that the system can be explained, tested, and improved in a structured way. The design also considers practical deployment requirements such as migrations, secure configuration, logging, data export, and administrator monitoring.

In this part of the project, the design decisions are connected to practical implementation. The section describes the role of the module, its interaction with other modules, the data it uses, and the expected result produced for users or administrators.

3.4.3 ECONOMICAL FEASIBILITY

3.5.3 Economic Feasibility is an important part of the DayTone project report. This section explains how the selected concept supports the overall objective of creating a secure, privacy-aware, AI-assisted mental wellness tracker. DayTone is designed with emphasis on privacy, usability, maintainability, and academic clarity. The implementation uses modular Flask blueprints, SQLAlchemy models, local NLP processing, and Scikit-learn prediction so that the system can be explained, tested, and improved in a structured way. The design also considers practical deployment requirements such as migrations, secure configuration, logging, data export, and administrator monitoring.

In this part of the project, the design decisions are connected to practical implementation. The section describes the role of the module, its interaction with other modules, the data it uses, and the expected result produced for users or administrators.

3.4.4 BEHAVIORAL FEASIBILITY

3.5.4 Behavioural Feasibility is an important part of the DayTone project report. This section explains how the selected concept supports the overall objective of creating a secure, privacy-aware, AI-assisted mental wellness tracker. DayTone is designed with emphasis on privacy, usability, maintainability, and academic clarity. The implementation uses modular Flask blueprints, SQLAlchemy models, local NLP processing, and Scikit-learn prediction so that the system can be explained, tested, and improved in a structured way. The design also considers practical deployment requirements such as migrations, secure configuration, logging, data export, and administrator monitoring.

In this part of the project, the design decisions are connected to practical implementation. The section describes the role of the module, its interaction with other modules, the data it uses, and the expected result produced for users or administrators.

CHAPTER 4
OPERATING ENVIRONMENT

4.1 REQUIREMENTS

4.1 Requirements is an important part of the DayTone project report. This section explains how the selected concept supports the overall objective of creating a secure, privacy-aware, AI-assisted mental wellness tracker. DayTone is designed with emphasis on privacy, usability, maintainability, and academic clarity. The implementation uses modular Flask blueprints, SQLAlchemy models, local NLP processing, and Scikit-learn prediction so that the system can be explained, tested, and improved in a structured way. The design also considers practical deployment requirements such as migrations, secure configuration, logging, data export, and administrator monitoring.

In this part of the project, the design decisions are connected to practical implementation. The section describes the role of the module, its interaction with other modules, the data it uses, and the expected result produced for users or administrators.

4.1.1 HARDWARE REQUIREMENTS

4.1.1 Hardware Requirements is an important part of the DayTone project report. This section explains how the selected concept supports the overall objective of creating a secure, privacy-aware, AI-assisted mental wellness tracker. DayTone is designed with emphasis on privacy, usability, maintainability, and academic clarity. The implementation uses modular Flask blueprints, SQLAlchemy models, local NLP processing, and Scikit-learn prediction so that the system can be explained, tested, and improved in a structured way. The design also considers practical deployment requirements such as migrations, secure configuration, logging, data export, and administrator monitoring.

In this part of the project, the design decisions are connected to practical implementation. The section describes the role of the module, its interaction with other modules, the data it uses, and the expected result produced for users or administrators.

4.1.2 SOFTWARE REQUIREMENTS

4.1.2 Software Requirements is an important part of the DayTone project report. This section explains how the selected concept supports the overall objective of creating a secure, privacy-aware, AI-assisted mental wellness tracker. DayTone is designed with emphasis on privacy, usability, maintainability, and academic clarity. The implementation uses modular Flask blueprints, SQLAlchemy models, local NLP processing, and Scikit-learn prediction so that the system can be explained, tested, and improved in a structured way. The design also considers practical deployment requirements such as migrations, secure configuration, logging, data export, and administrator monitoring.

In this part of the project, the design decisions are connected to practical implementation. The section describes the role of the module, its interaction with other modules, the data it uses, and the expected result produced for users or administrators.

CHAPTER 5
DESIGN

5.1 SYSTEM DESIGN

5.1 System Design is an important part of the DayTone project report. This section explains how the selected concept supports the overall objective of creating a secure, privacy-aware, AI-assisted mental wellness tracker. DayTone is designed with emphasis on privacy, usability, maintainability, and academic clarity. The implementation uses modular Flask blueprints, SQLAlchemy models, local NLP processing, and Scikit-learn prediction so that the system can be explained, tested, and improved in a structured way. The design also considers practical deployment requirements such as migrations, secure configuration, logging, data export, and administrator monitoring.

In this part of the project, the design decisions are connected to practical implementation. The section describes the role of the module, its interaction with other modules, the data it uses, and the expected result produced for users or administrators.

The same design philosophy is followed throughout DayTone: keep sensitive data protected, keep the interface simple, keep prediction results explainable, and keep the codebase maintainable for future enhancement.

In DayTone, 5.1 System Design is treated as a practical part of the complete wellness tracking workflow. The project does not present this topic only as a theoretical concept it connects the topic with actual screens, database records, backend routes, model outputs, and user decisions. This makes the report useful for explaining how the application works internally and how each module contributes to the final user experience.

The implementation follows a privacy-aware design approach. User wellness data may include personal habits, emotional reflections, stress values, and sleep patterns, so every major module is planned with confidentiality and controlled access in mind. Authentication, role-based pages, optional encryption, CSRF protection, and audit records support this requirement throughout the application.

From a software engineering perspective, the topic is implemented using modular Flask blueprints and reusable helper utilities. This makes the project easier to test and maintain because authentication, mood logging, administration, prediction, reports, mail, suggestions, and analytics are kept in separate areas of the codebase. A modular structure also makes future enhancement possible without rewriting the entire system.

From the user perspective, the topic is important because DayTone must remain simple and understandable. The system converts technical analysis into visible outputs such as dashboard cards, charts, heatmaps, risk labels, suggestions, history tables, and downloadable reports. The user is not expected to understand machine learning internals in order to benefit from the result.

In DayTone, 5.1 System Design is treated as a practical part of the complete wellness tracking workflow. The project does not present this topic only as a theoretical concept it connects the topic with actual screens, database records, backend routes, model outputs, and user decisions. This makes the report useful for explaining how the application works internally and how each module contributes to the final user experience.

5.1 SYSTEM DESIGN (CONTINUED)

This continuation page expands the previous discussion with additional implementation details, academic observations, and practical DayTone examples.

From a software engineering perspective, the topic is implemented using modular Flask blueprints and reusable helper utilities. This makes the project easier to test and maintain because authentication, mood logging, administration, prediction, reports, mail, suggestions, and analytics are kept in separate areas of the codebase. A modular structure also makes future enhancement possible without rewriting the entire system.

From the user perspective, the topic is important because DayTone must remain simple and understandable. The system converts technical analysis into visible outputs such as dashboard cards, charts, heatmaps, risk labels, suggestions, history tables, and downloadable reports. The user is not expected to understand machine learning internals in order to benefit from the result.

In DayTone, 5.1 System Design is treated as a practical part of the complete wellness tracking workflow. The project does not present this topic only as a theoretical concept it connects the topic with actual screens, database records, backend routes, model outputs, and user decisions. This makes the report useful for explaining how the application works internally and how each module contributes to the final user experience.

The implementation follows a privacy-aware design approach. User wellness data may include personal habits, emotional reflections, stress values, and sleep patterns, so every major module is planned with confidentiality and controlled access in mind. Authentication, role-based pages, optional encryption, CSRF protection, and audit records support this requirement throughout the application.

From a software engineering perspective, the topic is implemented using modular Flask blueprints and reusable helper utilities. This makes the project easier to test and maintain because authentication, mood logging, administration, prediction, reports, mail, suggestions, and analytics are kept in separate areas of the codebase. A modular structure also makes future enhancement possible without rewriting the entire system.

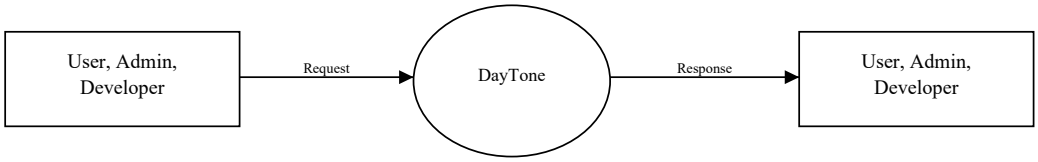
From the user perspective, the topic is important because DayTone must remain simple and understandable. The system converts technical analysis into visible outputs such as dashboard cards, charts, heatmaps, risk labels, suggestions, history tables, and downloadable reports. The user is not expected to understand machine learning internals in order to benefit from the result.

From the academic perspective, the topic demonstrates how web development, database management, data visualization, natural language processing, machine learning, security, and documentation can be combined into a single MCA project. It also shows awareness of limitations, especially the fact that the burnout model is trained on synthetic and semi-real validation data and is not a clinical diagnostic system.

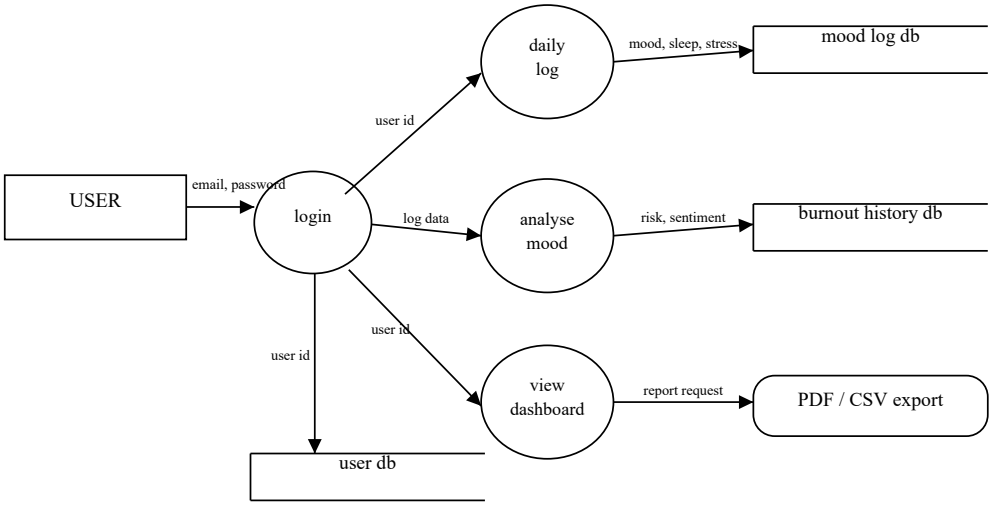
5.2 DATA FLOW DIAGRAM

The following data flow diagrams describe how information moves through DayTone. The diagrams follow standard DFD rules by separating external entities, processes, data stores, and labelled data flows.

Level 0 (Context Level)



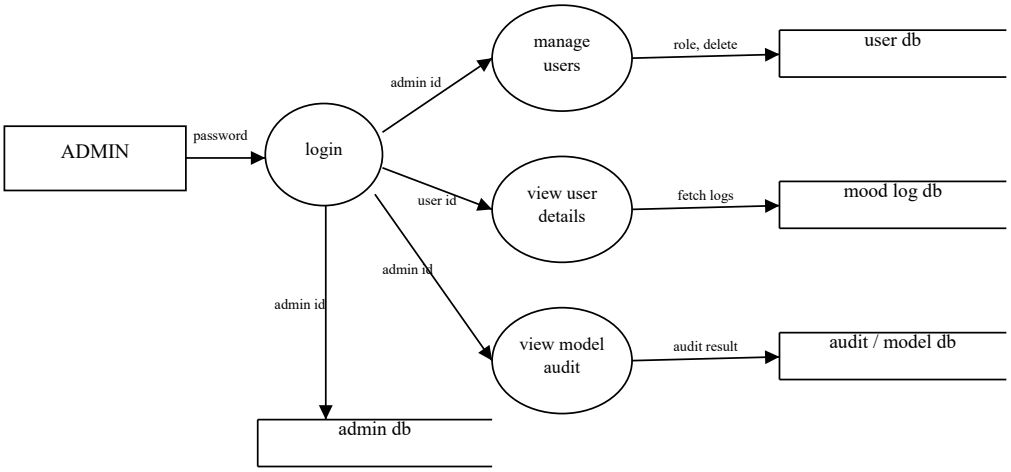
Level 1 (user)



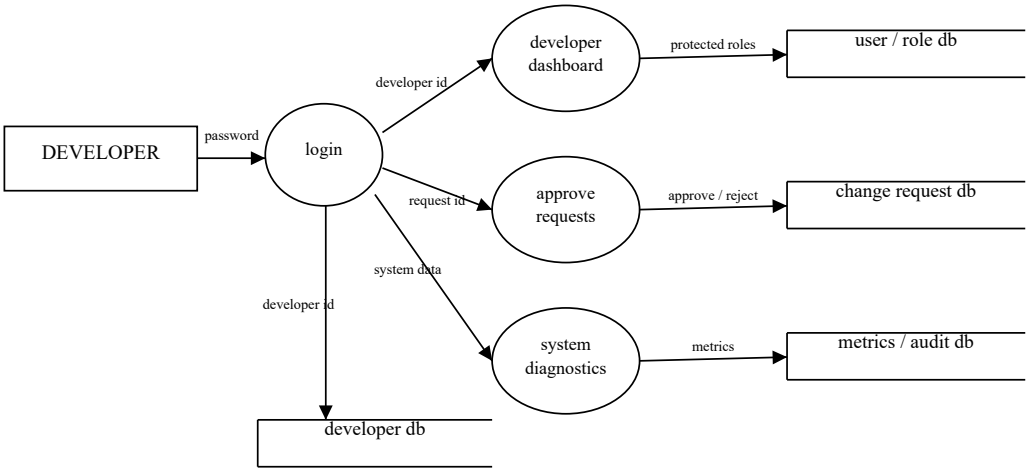
5.2 DATA FLOW DIAGRAM (CONTINUED)

The Level 1 diagrams are separated by role because DayTone contains normal user workflows, administrator workflows, and a protected developer workflow. This makes the DFD closer to the actual project structure.

Level 1 (admin)



Level 1 (developer)



5.3 INPUT DESIGN

5.3 Input Design is an important part of the DayTone project report. This section explains how the selected concept supports the overall objective of creating a secure, privacy-aware, AI-assisted mental wellness tracker. DayTone is designed with emphasis on privacy, usability, maintainability, and academic clarity. The implementation uses modular Flask blueprints, SQLAlchemy models, local NLP processing, and Scikit-learn prediction so that the system can be explained, tested, and improved in a structured way. The design also considers practical deployment requirements such as migrations, secure configuration, logging, data export, and administrator monitoring.

In this part of the project, the design decisions are connected to practical implementation. The section describes the role of the module, its interaction with other modules, the data it uses, and the expected result produced for users or administrators.

The same design philosophy is followed throughout DayTone: keep sensitive data protected, keep the interface simple, keep prediction results explainable, and keep the codebase maintainable for future enhancement.

In DayTone, 5.3 Input Design is treated as a practical part of the complete wellness tracking workflow. The project does not present this topic only as a theoretical concept it connects the topic with actual screens, database records, backend routes, model outputs, and user decisions. This makes the report useful for explaining how the application works internally and how each module contributes to the final user experience.

The implementation follows a privacy-aware design approach. User wellness data may include personal habits, emotional reflections, stress values, and sleep patterns, so every major module is planned with confidentiality and controlled access in mind. Authentication, role-based pages, optional encryption, CSRF protection, and audit records support this requirement throughout the application.

From a software engineering perspective, the topic is implemented using modular Flask blueprints and reusable helper utilities. This makes the project easier to test and maintain because authentication, mood logging, administration, prediction, reports, mail, suggestions, and analytics are kept in separate areas of the codebase. A modular structure also makes future enhancement possible without rewriting the entire system.

From the user perspective, the topic is important because DayTone must remain simple and understandable. The system converts technical analysis into visible outputs such as dashboard cards, charts, heatmaps, risk labels, suggestions, history tables, and downloadable reports. The user is not expected to understand machine learning internals in order to benefit from the result.

In DayTone, 5.3 Input Design is treated as a practical part of the complete wellness tracking workflow. The project does not present this topic only as a theoretical concept it connects the topic with actual screens, database records, backend routes, model outputs, and user decisions. This makes the report useful for explaining how the application works internally and how each module contributes to the final user experience.

5.3 INPUT DESIGN (CONTINUED)

This continuation page expands the previous discussion with additional implementation details, academic observations, and practical DayTone examples.

From a software engineering perspective, the topic is implemented using modular Flask blueprints and reusable helper utilities. This makes the project easier to test and maintain because authentication, mood logging, administration, prediction, reports, mail, suggestions, and analytics are kept in separate areas of the codebase. A modular structure also makes future enhancement possible without rewriting the entire system.

From the user perspective, the topic is important because DayTone must remain simple and understandable. The system converts technical analysis into visible outputs such as dashboard cards, charts, heatmaps, risk labels, suggestions, history tables, and downloadable reports. The user is not expected to understand machine learning internals in order to benefit from the result.

In DayTone, 5.3 Input Design is treated as a practical part of the complete wellness tracking workflow. The project does not present this topic only as a theoretical concept it connects the topic with actual screens, database records, backend routes, model outputs, and user decisions. This makes the report useful for explaining how the application works internally and how each module contributes to the final user experience.

The implementation follows a privacy-aware design approach. User wellness data may include personal habits, emotional reflections, stress values, and sleep patterns, so every major module is planned with confidentiality and controlled access in mind. Authentication, role-based pages, optional encryption, CSRF protection, and audit records support this requirement throughout the application.

From a software engineering perspective, the topic is implemented using modular Flask blueprints and reusable helper utilities. This makes the project easier to test and maintain because authentication, mood logging, administration, prediction, reports, mail, suggestions, and analytics are kept in separate areas of the codebase. A modular structure also makes future enhancement possible without rewriting the entire system.

From the user perspective, the topic is important because DayTone must remain simple and understandable. The system converts technical analysis into visible outputs such as dashboard cards, charts, heatmaps, risk labels, suggestions, history tables, and downloadable reports. The user is not expected to understand machine learning internals in order to benefit from the result.

From the academic perspective, the topic demonstrates how web development, database management, data visualization, natural language processing, machine learning, security, and documentation can be combined into a single MCA project. It also shows awareness of limitations, especially the fact that the burnout model is trained on synthetic and semi-real validation data and is not a clinical diagnostic system.

5.4 OUTPUT DESIGN

5.4 Output Design is an important part of the DayTone project report. This section explains how the selected concept supports the overall objective of creating a secure, privacy-aware, AI-assisted mental wellness tracker. DayTone is designed with emphasis on privacy, usability, maintainability, and academic clarity. The implementation uses modular Flask blueprints, SQLAlchemy models, local NLP processing, and Scikit-learn prediction so that the system can be explained, tested, and improved in a structured way. The design also considers practical deployment requirements such as migrations, secure configuration, logging, data export, and administrator monitoring.

In this part of the project, the design decisions are connected to practical implementation. The section describes the role of the module, its interaction with other modules, the data it uses, and the expected result produced for users or administrators.

The same design philosophy is followed throughout DayTone: keep sensitive data protected, keep the interface simple, keep prediction results explainable, and keep the codebase maintainable for future enhancement.

In DayTone, 5.4 Output Design is treated as a practical part of the complete wellness tracking workflow. The project does not present this topic only as a theoretical concept it connects the topic with actual screens, database records, backend routes, model outputs, and user decisions. This makes the report useful for explaining how the application works internally and how each module contributes to the final user experience.

The implementation follows a privacy-aware design approach. User wellness data may include personal habits, emotional reflections, stress values, and sleep patterns, so every major module is planned with confidentiality and controlled access in mind. Authentication, role-based pages, optional encryption, CSRF protection, and audit records support this requirement throughout the application.

From a software engineering perspective, the topic is implemented using modular Flask blueprints and reusable helper utilities. This makes the project easier to test and maintain because authentication, mood logging, administration, prediction, reports, mail, suggestions, and analytics are kept in separate areas of the codebase. A modular structure also makes future enhancement possible without rewriting the entire system.

From the user perspective, the topic is important because DayTone must remain simple and understandable. The system converts technical analysis into visible outputs such as dashboard cards, charts, heatmaps, risk labels, suggestions, history tables, and downloadable reports. The user is not expected to understand machine learning internals in order to benefit from the result.

In DayTone, 5.4 Output Design is treated as a practical part of the complete wellness tracking workflow. The project does not present this topic only as a theoretical concept it connects the topic with actual screens, database records, backend routes, model outputs, and user decisions. This makes the report useful for explaining how the application works internally and how each module contributes to the final user experience.

5.4 OUTPUT DESIGN (CONTINUED)

This continuation page expands the previous discussion with additional implementation details, academic observations, and practical DayTone examples.

From a software engineering perspective, the topic is implemented using modular Flask blueprints and reusable helper utilities. This makes the project easier to test and maintain because authentication, mood logging, administration, prediction, reports, mail, suggestions, and analytics are kept in separate areas of the codebase. A modular structure also makes future enhancement possible without rewriting the entire system.

From the user perspective, the topic is important because DayTone must remain simple and understandable. The system converts technical analysis into visible outputs such as dashboard cards, charts, heatmaps, risk labels, suggestions, history tables, and downloadable reports. The user is not expected to understand machine learning internals in order to benefit from the result.

In DayTone, 5.4 Output Design is treated as a practical part of the complete wellness tracking workflow. The project does not present this topic only as a theoretical concept it connects the topic with actual screens, database records, backend routes, model outputs, and user decisions. This makes the report useful for explaining how the application works internally and how each module contributes to the final user experience.

The implementation follows a privacy-aware design approach. User wellness data may include personal habits, emotional reflections, stress values, and sleep patterns, so every major module is planned with confidentiality and controlled access in mind. Authentication, role-based pages, optional encryption, CSRF protection, and audit records support this requirement throughout the application.

From a software engineering perspective, the topic is implemented using modular Flask blueprints and reusable helper utilities. This makes the project easier to test and maintain because authentication, mood logging, administration, prediction, reports, mail, suggestions, and analytics are kept in separate areas of the codebase. A modular structure also makes future enhancement possible without rewriting the entire system.

From the user perspective, the topic is important because DayTone must remain simple and understandable. The system converts technical analysis into visible outputs such as dashboard cards, charts, heatmaps, risk labels, suggestions, history tables, and downloadable reports. The user is not expected to understand machine learning internals in order to benefit from the result.

From the academic perspective, the topic demonstrates how web development, database management, data visualization, natural language processing, machine learning, security, and documentation can be combined into a single MCA project. It also shows awareness of limitations, especially the fact that the burnout model is trained on synthetic and semi-real validation data and is not a clinical diagnostic system.

CHAPTER 6
FUNCTIONAL & NON-FUNCTIONAL REQUIREMENTS

6.1 FUNCTIONAL REQUIREMENTS

6.1 Functional Requirements is an important part of the DayTone project report. This section explains how the selected concept supports the overall objective of creating a secure, privacy-aware, AI-assisted mental wellness tracker. DayTone is designed with emphasis on privacy, usability, maintainability, and academic clarity. The implementation uses modular Flask blueprints, SQLAlchemy models, local NLP processing, and Scikit-learn prediction so that the system can be explained, tested, and improved in a structured way. The design also considers practical deployment requirements such as migrations, secure configuration, logging, data export, and administrator monitoring.

In this part of the project, the design decisions are connected to practical implementation. The section describes the role of the module, its interaction with other modules, the data it uses, and the expected result produced for users or administrators.

The same design philosophy is followed throughout DayTone: keep sensitive data protected, keep the interface simple, keep prediction results explainable, and keep the codebase maintainable for future enhancement.

In DayTone, 6.1 Functional Requirements is treated as a practical part of the complete wellness tracking workflow. The project does not present this topic only as a theoretical concept it connects the topic with actual screens, database records, backend routes, model outputs, and user decisions. This makes the report useful for explaining how the application works internally and how each module contributes to the final user experience.

The implementation follows a privacy-aware design approach. User wellness data may include personal habits, emotional reflections, stress values, and sleep patterns, so every major module is planned with confidentiality and controlled access in mind. Authentication, role-based pages, optional encryption, CSRF protection, and audit records support this requirement throughout the application.

From a software engineering perspective, the topic is implemented using modular Flask blueprints and reusable helper utilities. This makes the project easier to test and maintain because authentication, mood logging, administration, prediction, reports, mail, suggestions, and analytics are kept in separate areas of the codebase. A modular structure also makes future enhancement possible without rewriting the entire system.

From the user perspective, the topic is important because DayTone must remain simple and understandable. The system converts technical analysis into visible outputs such as dashboard cards, charts, heatmaps, risk labels, suggestions, history tables, and downloadable reports. The user is not expected to understand machine learning internals in order to benefit from the result.

In DayTone, 6.1 Functional Requirements is treated as a practical part of the complete wellness tracking workflow. The project does not present this topic only as a theoretical concept it connects the topic with actual screens, database records, backend routes, model outputs, and user decisions. This makes the report useful for explaining how the application works internally and how each module contributes to the final user experience.

6.2 NON - FUNCTIONAL REQUIREMENTS

6.2 Non - Functional Requirements is an important part of the DayTone project report. This section explains how the selected concept supports the overall objective of creating a secure, privacy-aware, AI-assisted mental wellness tracker. DayTone is designed with emphasis on privacy, usability, maintainability, and academic clarity. The implementation uses modular Flask blueprints, SQLAlchemy models, local NLP processing, and Scikit-learn prediction so that the system can be explained, tested, and improved in a structured way. The design also considers practical deployment requirements such as migrations, secure configuration, logging, data export, and administrator monitoring.

In this part of the project, the design decisions are connected to practical implementation. The section describes the role of the module, its interaction with other modules, the data it uses, and the expected result produced for users or administrators.

The same design philosophy is followed throughout DayTone: keep sensitive data protected, keep the interface simple, keep prediction results explainable, and keep the codebase maintainable for future enhancement.

In DayTone, 6.2 Non Functional Requirements is treated as a practical part of the complete wellness tracking workflow. The project does not present this topic only as a theoretical concept it connects the topic with actual screens, database records, backend routes, model outputs, and user decisions. This makes the report useful for explaining how the application works internally and how each module contributes to the final user experience.

The implementation follows a privacy-aware design approach. User wellness data may include personal habits, emotional reflections, stress values, and sleep patterns, so every major module is planned with confidentiality and controlled access in mind. Authentication, role-based pages, optional encryption, CSRF protection, and audit records support this requirement throughout the application.

From a software engineering perspective, the topic is implemented using modular Flask blueprints and reusable helper utilities. This makes the project easier to test and maintain because authentication, mood logging, administration, prediction, reports, mail, suggestions, and analytics are kept in separate areas of the codebase. A modular structure also makes future enhancement possible without rewriting the entire system.

From the user perspective, the topic is important because DayTone must remain simple and understandable. The system converts technical analysis into visible outputs such as dashboard cards, charts, heatmaps, risk labels, suggestions, history tables, and downloadable reports. The user is not expected to understand machine learning internals in order to benefit from the result.

In DayTone, 6.2 Non Functional Requirements is treated as a practical part of the complete wellness tracking workflow. The project does not present this topic only as a theoretical concept it connects the topic with actual screens, database records, backend routes, model outputs, and user decisions. This makes the report useful for explaining how the application works internally and how each module contributes to the final user experience.

6.2 NON - FUNCTIONAL REQUIREMENTS (CONTINUED)

This continuation page expands the previous discussion with additional implementation details, academic observations, and practical DayTone examples.

From a software engineering perspective, the topic is implemented using modular Flask blueprints and reusable helper utilities. This makes the project easier to test and maintain because authentication, mood logging, administration, prediction, reports, mail, suggestions, and analytics are kept in separate areas of the codebase. A modular structure also makes future enhancement possible without rewriting the entire system.

From the user perspective, the topic is important because DayTone must remain simple and understandable. The system converts technical analysis into visible outputs such as dashboard cards, charts, heatmaps, risk labels, suggestions, history tables, and downloadable reports. The user is not expected to understand machine learning internals in order to benefit from the result.

In DayTone, 6.2 Non Functional Requirements is treated as a practical part of the complete wellness tracking workflow. The project does not present this topic only as a theoretical concept it connects the topic with actual screens, database records, backend routes, model outputs, and user decisions. This makes the report useful for explaining how the application works internally and how each module contributes to the final user experience.

The implementation follows a privacy-aware design approach. User wellness data may include personal habits, emotional reflections, stress values, and sleep patterns, so every major module is planned with confidentiality and controlled access in mind. Authentication, role-based pages, optional encryption, CSRF protection, and audit records support this requirement throughout the application.

From a software engineering perspective, the topic is implemented using modular Flask blueprints and reusable helper utilities. This makes the project easier to test and maintain because authentication, mood logging, administration, prediction, reports, mail, suggestions, and analytics are kept in separate areas of the codebase. A modular structure also makes future enhancement possible without rewriting the entire system.

From the user perspective, the topic is important because DayTone must remain simple and understandable. The system converts technical analysis into visible outputs such as dashboard cards, charts, heatmaps, risk labels, suggestions, history tables, and downloadable reports. The user is not expected to understand machine learning internals in order to benefit from the result.

From the academic perspective, the topic demonstrates how web development, database management, data visualization, natural language processing, machine learning, security, and documentation can be combined into a single MCA project. It also shows awareness of limitations, especially the fact that the burnout model is trained on synthetic and semi-real validation data and is not a clinical diagnostic system.

CHAPTER 7
ALGORITHM

7.1 DECISION TREE ALGORITHM (ML MODEL)

Decision Tree is used in DayTone as the currently selected machine learning model because the training pipeline compares Decision Tree, Logistic Regression, and Random Forest, then saves the model with the best holdout accuracy. The active model metadata identifies DecisionTree as the saved burnout prediction algorithm.

A Decision Tree classifier works by splitting the input data into branches based on feature values. In DayTone, the tree studies wellness indicators such as mood score, sleep hours, stress level, activity status, social interaction, sentiment score, and rolling seven-day values. These splits lead to a final class label such as Low, Medium, or High burnout risk.

This algorithm is suitable for an academic wellness tracker because its decision process is easier to explain than many black-box models. The system can connect a prediction with understandable drivers such as low mood, high stress, reduced sleep, negative journal sentiment, or repeated difficult days.

The DayTone feature vector includes current mood, sleep hours, stress level, activity status, social interaction, sentiment score, seven-day average mood, seven-day average stress, seven-day average sleep, count of repeated difficult days, and similar behavioural indicators. These values allow the model to study both the current day and the recent trend.

The model output is intentionally simple: Low, Medium, or High burnout risk. This makes the prediction understandable for users and suitable for dashboard display. The system avoids presenting the value as a medical diagnosis and treats it as an early awareness signal.

The ML pipeline is trained with synthetic academic data and can later be retrained using exported real logs after proper consent and anonymisation. This keeps the current project privacy-safe while leaving a clear future path for accuracy improvement.

7.1 Decision Tree Algorithm (Ml Model) is an important part of the DayTone project report. This section explains how the selected concept supports the overall objective of creating a secure, privacy-aware, AI-assisted mental wellness tracker. DayTone is designed with emphasis on privacy, usability, maintainability, and academic clarity. The implementation uses modular Flask blueprints, SQLAlchemy models, local NLP processing, and Scikit-learn prediction so that the system can be explained, tested, and improved in a structured way. The design also considers practical deployment requirements such as migrations, secure configuration, logging, data export, and administrator monitoring.

In this part of the project, the design decisions are connected to practical implementation. The section describes the role of the module, its interaction with other modules, the data it uses, and the expected result produced for users or administrators.

The same design philosophy is followed throughout DayTone: keep sensitive data protected, keep the interface simple, keep prediction results explainable, and keep the codebase maintainable for future enhancement.

CHAPTER 8
TESTING

8.1 SYSTEM TESTING

8.1 System Testing is an important part of the DayTone project report. This section explains how the selected concept supports the overall objective of creating a secure, privacy-aware, AI-assisted mental wellness tracker. DayTone is designed with emphasis on privacy, usability, maintainability, and academic clarity. The implementation uses modular Flask blueprints, SQLAlchemy models, local NLP processing, and Scikit-learn prediction so that the system can be explained, tested, and improved in a structured way. The design also considers practical deployment requirements such as migrations, secure configuration, logging, data export, and administrator monitoring.

In this part of the project, the design decisions are connected to practical implementation. The section describes the role of the module, its interaction with other modules, the data it uses, and the expected result produced for users or administrators.

8.2 UNIT TESTING

8.2 Unit Testing is an important part of the DayTone project report. This section explains how the selected concept supports the overall objective of creating a secure, privacy-aware, AI-assisted mental wellness tracker. DayTone is designed with emphasis on privacy, usability, maintainability, and academic clarity. The implementation uses modular Flask blueprints, SQLAlchemy models, local NLP processing, and Scikit-learn prediction so that the system can be explained, tested, and improved in a structured way. The design also considers practical deployment requirements such as migrations, secure configuration, logging, data export, and administrator monitoring.

In this part of the project, the design decisions are connected to practical implementation. The section describes the role of the module, its interaction with other modules, the data it uses, and the expected result produced for users or administrators.

8.3 INTEGRATION TESTING

8.3 Integration Testing is an important part of the DayTone project report. This section explains how the selected concept supports the overall objective of creating a secure, privacy-aware, AI-assisted mental wellness tracker. DayTone is designed with emphasis on privacy, usability, maintainability, and academic clarity. The implementation uses modular Flask blueprints, SQLAlchemy models, local NLP processing, and Scikit-learn prediction so that the system can be explained, tested, and improved in a structured way. The design also considers practical deployment requirements such as migrations, secure configuration, logging, data export, and administrator monitoring.

In this part of the project, the design decisions are connected to practical implementation. The section describes the role of the module, its interaction with other modules, the data it uses, and the expected result produced for users or administrators.

The same design philosophy is followed throughout DayTone: keep sensitive data protected, keep the interface simple, keep prediction results explainable, and keep the codebase maintainable for future enhancement.

In DayTone, 8.3 Integration Testing is treated as a practical part of the complete wellness tracking workflow. The project does not present this topic only as a theoretical concept it connects the topic with actual screens, database records, backend routes, model outputs, and user decisions. This makes the report useful for explaining how the application works internally and how each module contributes to the final user experience.

The implementation follows a privacy-aware design approach. User wellness data may include personal habits, emotional reflections, stress values, and sleep patterns, so every major module is planned with confidentiality and controlled access in mind. Authentication, role-based pages, optional encryption, CSRF protection, and audit records support this requirement throughout the application.

From a software engineering perspective, the topic is implemented using modular Flask blueprints and reusable helper utilities. This makes the project easier to test and maintain because authentication, mood logging, administration, prediction, reports, mail, suggestions, and analytics are kept in separate areas of the codebase. A modular structure also makes future enhancement possible without rewriting the entire system.

From the user perspective, the topic is important because DayTone must remain simple and understandable. The system converts technical analysis into visible outputs such as dashboard cards, charts, heatmaps, risk labels, suggestions, history tables, and downloadable reports. The user is not expected to understand machine learning internals in order to benefit from the result.

In DayTone, 8.3 Integration Testing is treated as a practical part of the complete wellness tracking workflow. The project does not present this topic only as a theoretical concept it connects the topic with actual screens, database records, backend routes, model outputs, and user decisions. This makes the report useful for explaining how the application works internally and how each module contributes to the final user experience.

8.4 BLACK BOX TESTING

8.4 Black Box Testing is an important part of the DayTone project report. This section explains how the selected concept supports the overall objective of creating a secure, privacy-aware, AI-assisted mental wellness tracker. DayTone is designed with emphasis on privacy, usability, maintainability, and academic clarity. The implementation uses modular Flask blueprints, SQLAlchemy models, local NLP processing, and Scikit-learn prediction so that the system can be explained, tested, and improved in a structured way. The design also considers practical deployment requirements such as migrations, secure configuration, logging, data export, and administrator monitoring.

In this part of the project, the design decisions are connected to practical implementation. The section describes the role of the module, its interaction with other modules, the data it uses, and the expected result produced for users or administrators.

8.5 VALIDATION TESTING

8.5 Validation Testing is an important part of the DayTone project report. This section explains how the selected concept supports the overall objective of creating a secure, privacy-aware, AI-assisted mental wellness tracker. DayTone is designed with emphasis on privacy, usability, maintainability, and academic clarity. The implementation uses modular Flask blueprints, SQLAlchemy models, local NLP processing, and Scikit-learn prediction so that the system can be explained, tested, and improved in a structured way. The design also considers practical deployment requirements such as migrations, secure configuration, logging, data export, and administrator monitoring.

In this part of the project, the design decisions are connected to practical implementation. The section describes the role of the module, its interaction with other modules, the data it uses, and the expected result produced for users or administrators.

8.6 OUTPUT TESTING

8.6 Output Testing is an important part of the DayTone project report. This section explains how the selected concept supports the overall objective of creating a secure, privacy-aware, AI-assisted mental wellness tracker. DayTone is designed with emphasis on privacy, usability, maintainability, and academic clarity. The implementation uses modular Flask blueprints, SQLAlchemy models, local NLP processing, and Scikit-learn prediction so that the system can be explained, tested, and improved in a structured way. The design also considers practical deployment requirements such as migrations, secure configuration, logging, data export, and administrator monitoring.

In this part of the project, the design decisions are connected to practical implementation. The section describes the role of the module, its interaction with other modules, the data it uses, and the expected result produced for users or administrators.

8.7 USER ACCEPTANCE TESTING

8.7 User Acceptance Testing is an important part of the DayTone project report. This section explains how the selected concept supports the overall objective of creating a secure, privacy-aware, AI-assisted mental wellness tracker. DayTone is designed with emphasis on privacy, usability, maintainability, and academic clarity. The implementation uses modular Flask blueprints, SQLAlchemy models, local NLP processing, and Scikit-learn prediction so that the system can be explained, tested, and improved in a structured way. The design also considers practical deployment requirements such as migrations, secure configuration, logging, data export, and administrator monitoring.

In this part of the project, the design decisions are connected to practical implementation. The section describes the role of the module, its interaction with other modules, the data it uses, and the expected result produced for users or administrators.

8.8 WHITE BOX TESTING

8.8 White Box Testing is an important part of the DayTone project report. This section explains how the selected concept supports the overall objective of creating a secure, privacy-aware, AI-assisted mental wellness tracker. DayTone is designed with emphasis on privacy, usability, maintainability, and academic clarity. The implementation uses modular Flask blueprints, SQLAlchemy models, local NLP processing, and Scikit-learn prediction so that the system can be explained, tested, and improved in a structured way. The design also considers practical deployment requirements such as migrations, secure configuration, logging, data export, and administrator monitoring.

In this part of the project, the design decisions are connected to practical implementation. The section describes the role of the module, its interaction with other modules, the data it uses, and the expected result produced for users or administrators.

The same design philosophy is followed throughout DayTone: keep sensitive data protected, keep the interface simple, keep prediction results explainable, and keep the codebase maintainable for future enhancement.

In DayTone, 8.8 White Box Testing is treated as a practical part of the complete wellness tracking workflow. The project does not present this topic only as a theoretical concept it connects the topic with actual screens, database records, backend routes, model outputs, and user decisions. This makes the report useful for explaining how the application works internally and how each module contributes to the final user experience.

The implementation follows a privacy-aware design approach. User wellness data may include personal habits, emotional reflections, stress values, and sleep patterns, so every major module is planned with confidentiality and controlled access in mind. Authentication, role-based pages, optional encryption, CSRF protection, and audit records support this requirement throughout the application.

From a software engineering perspective, the topic is implemented using modular Flask blueprints and reusable helper utilities. This makes the project easier to test and maintain because authentication, mood logging, administration, prediction, reports, mail, suggestions, and analytics are kept in separate areas of the codebase. A modular structure also makes future enhancement possible without rewriting the entire system.

From the user perspective, the topic is important because DayTone must remain simple and understandable. The system converts technical analysis into visible outputs such as dashboard cards, charts, heatmaps, risk labels, suggestions, history tables, and downloadable reports. The user is not expected to understand machine learning internals in order to benefit from the result.

In DayTone, 8.8 White Box Testing is treated as a practical part of the complete wellness tracking workflow. The project does not present this topic only as a theoretical concept it connects the topic with actual screens, database records, backend routes, model outputs, and user decisions. This makes the report useful for explaining how the application works internally and how each module contributes to the final user experience.

CHAPTER 9
APPENDICES

9.1 SCREENSHOTS

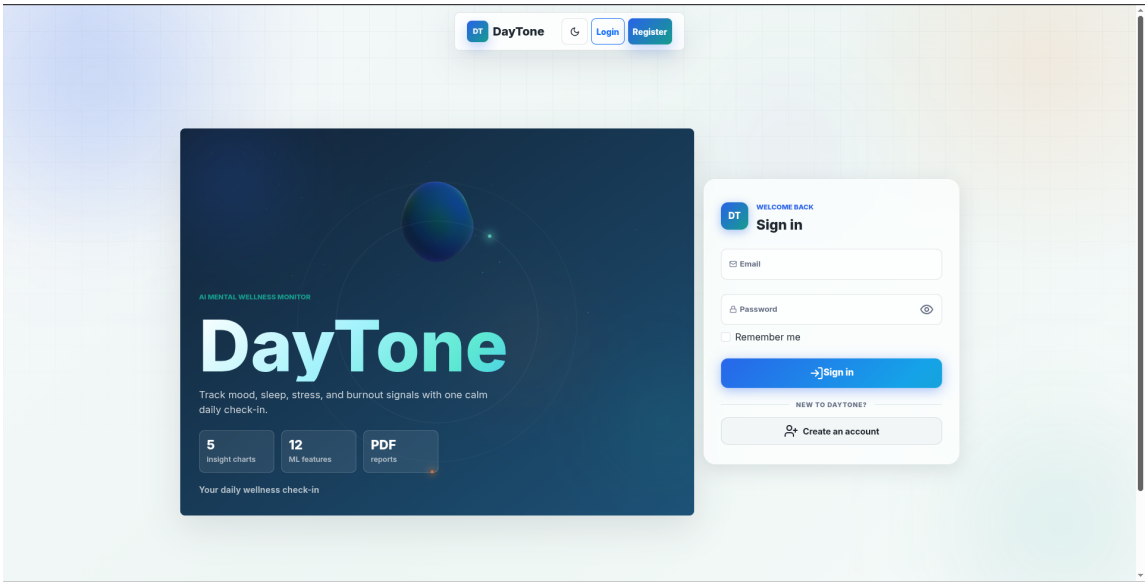


Figure 1: Login screen with DayTone welcome panel and sign-in form

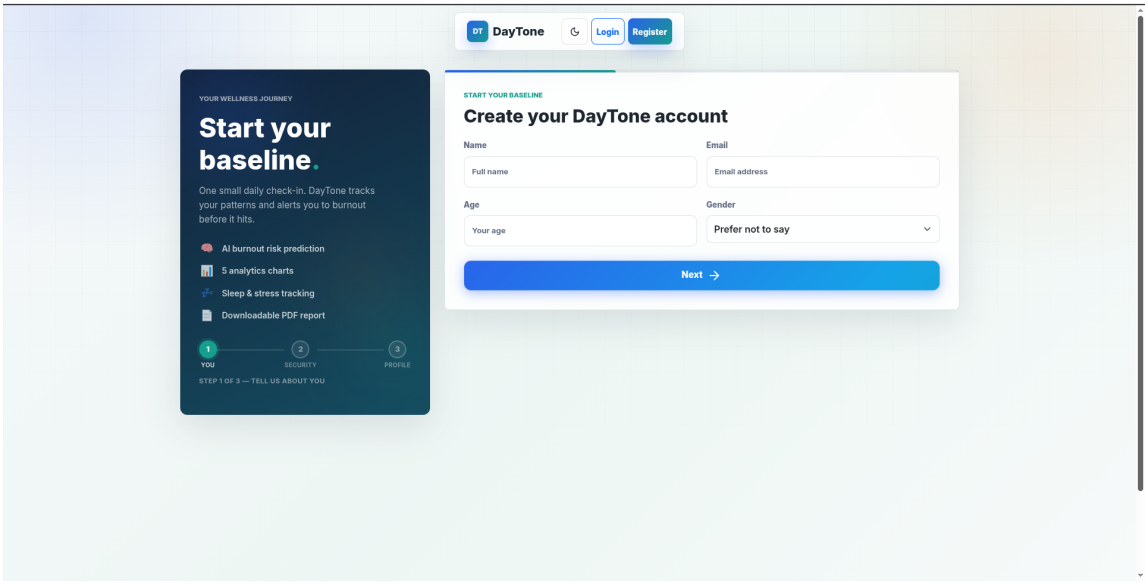


Figure 2: Registration screen for creating a DayTone account

9.1 SCREENSHOTS (CONTINUED)

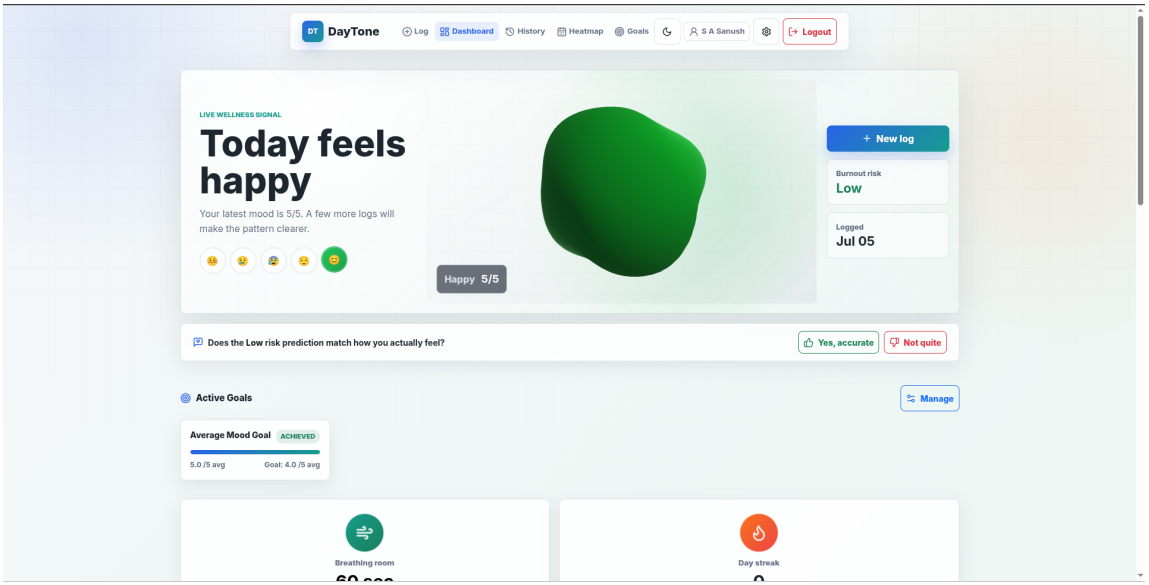


Figure 3: User dashboard showing current mood, burnout risk and daily status

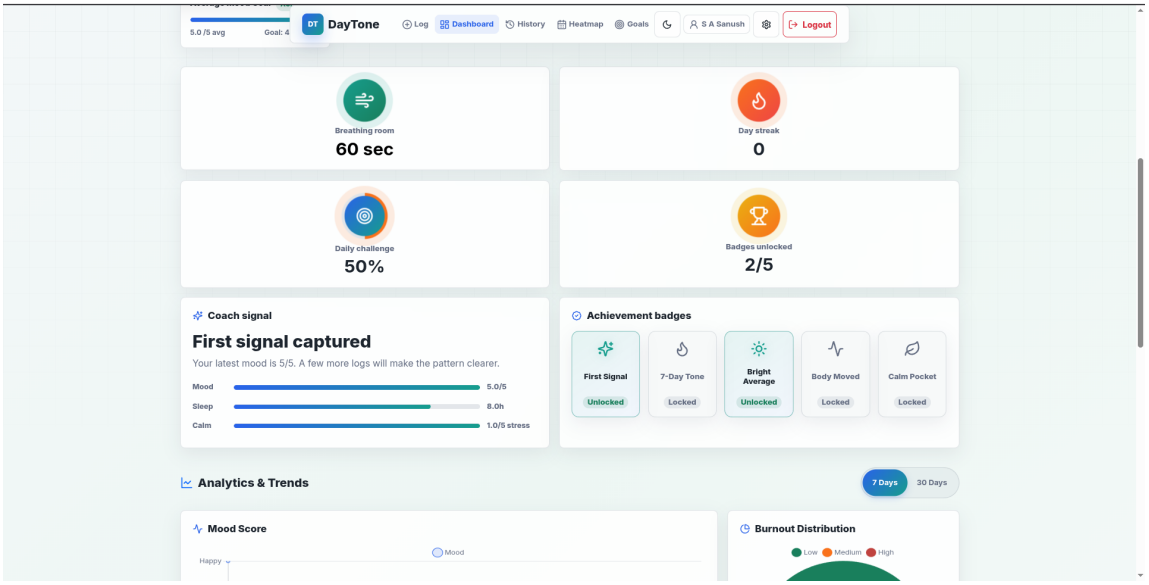


Figure 4: Dashboard wellness cards, coach signal and achievement badges

9.1 SCREENSHOTS (CONTINUED)

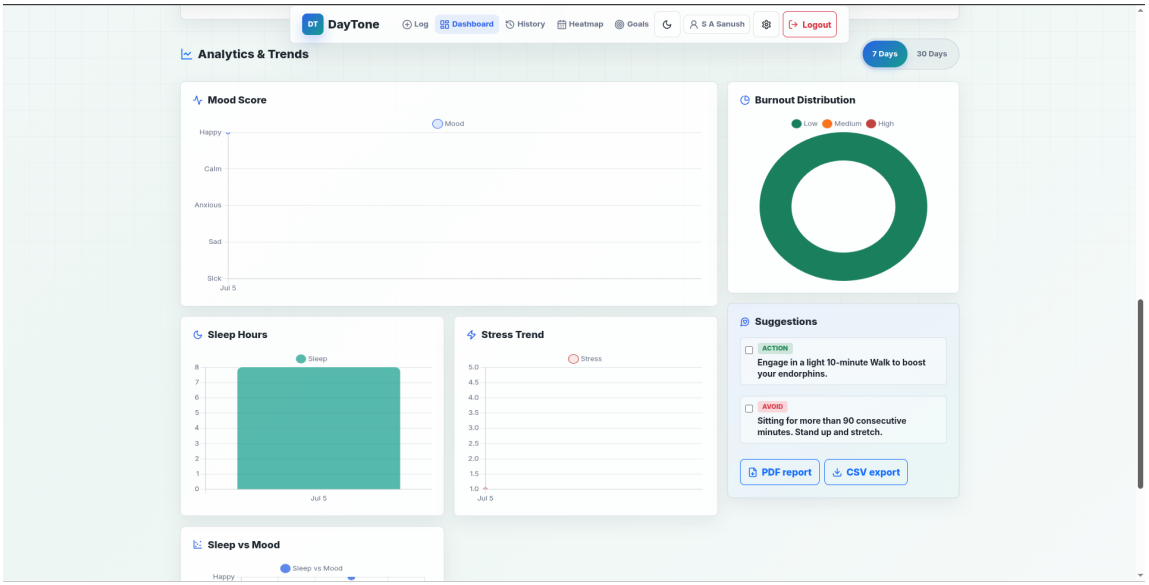


Figure 5: Analytics and trends section with mood, sleep, stress and suggestions

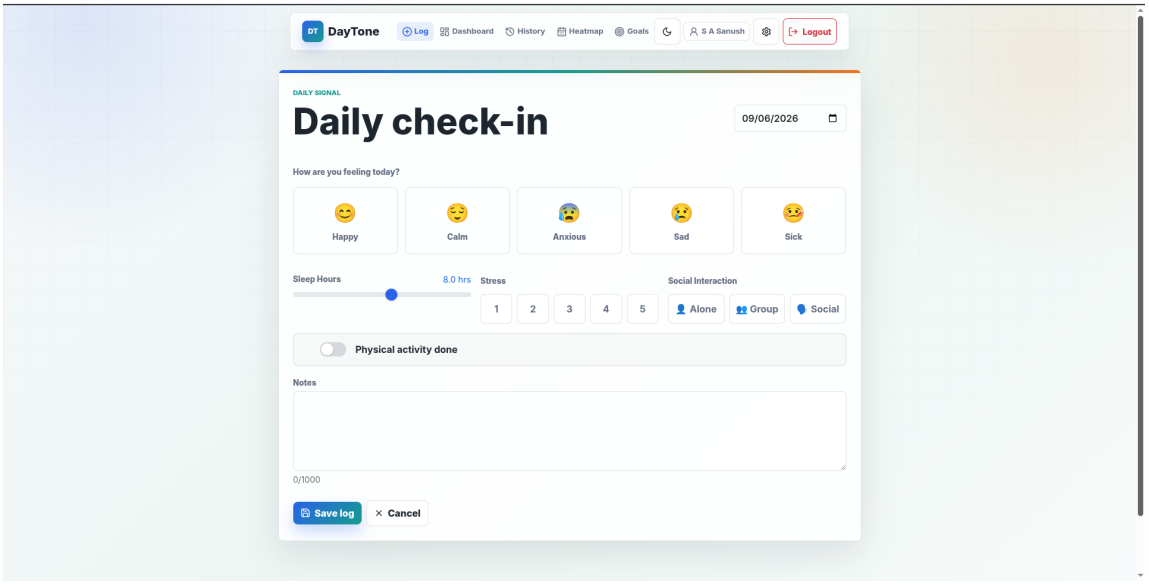


Figure 6: Daily check-in form for mood, sleep, stress, activity and notes

9.1 SCREENSHOTS (CONTINUED)

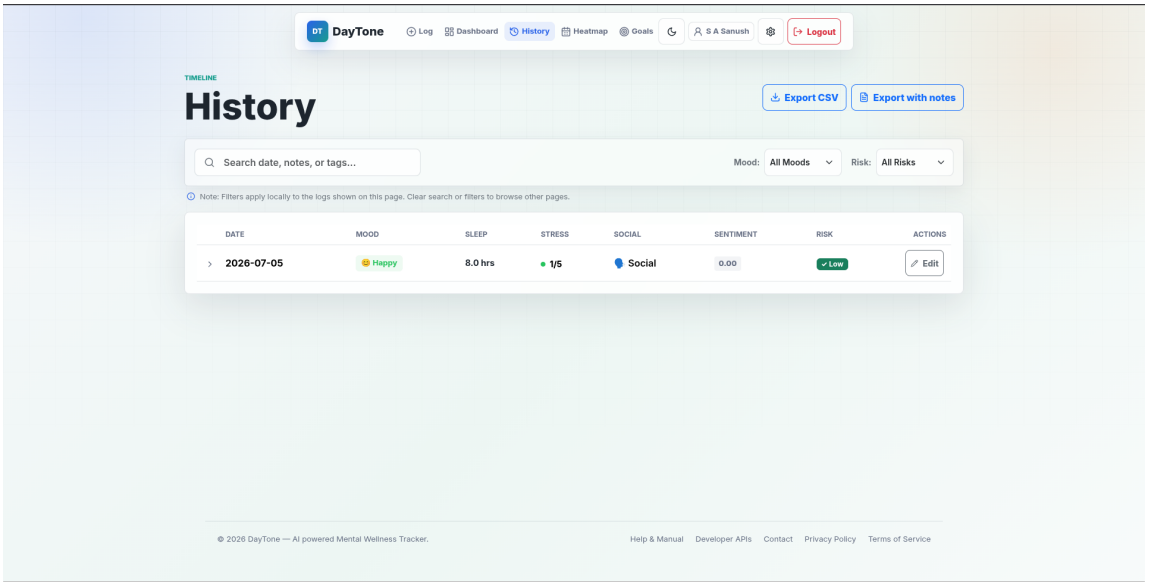


Figure 7: History screen with search, filters and export options

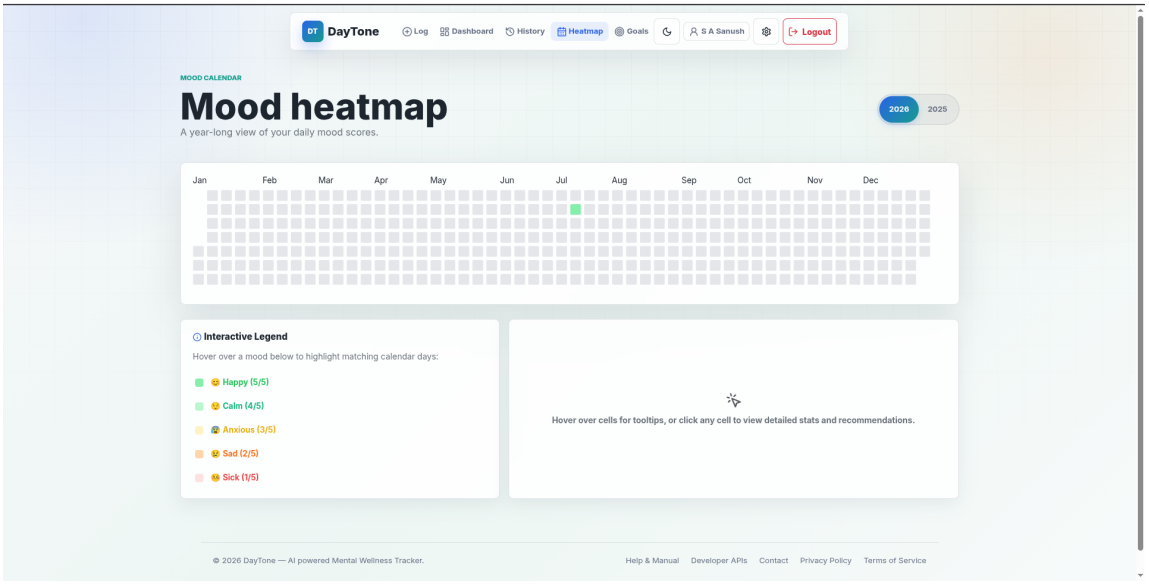


Figure 8: Mood heatmap showing yearly mood activity and legend

9.1 SCREENSHOTS (CONTINUED)

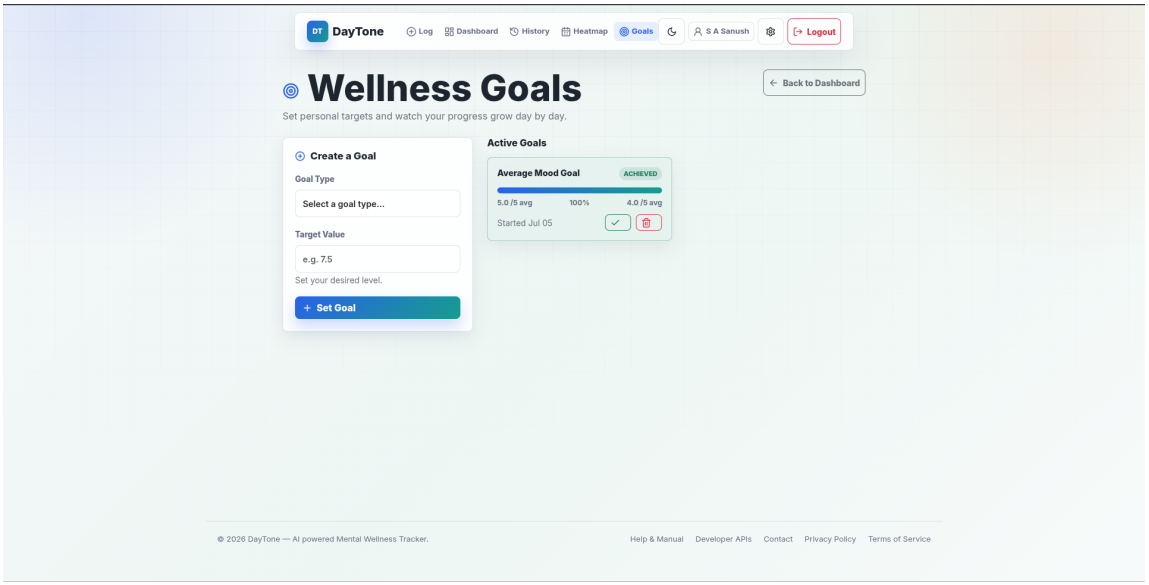


Figure 9: Wellness goals page with active goal progress

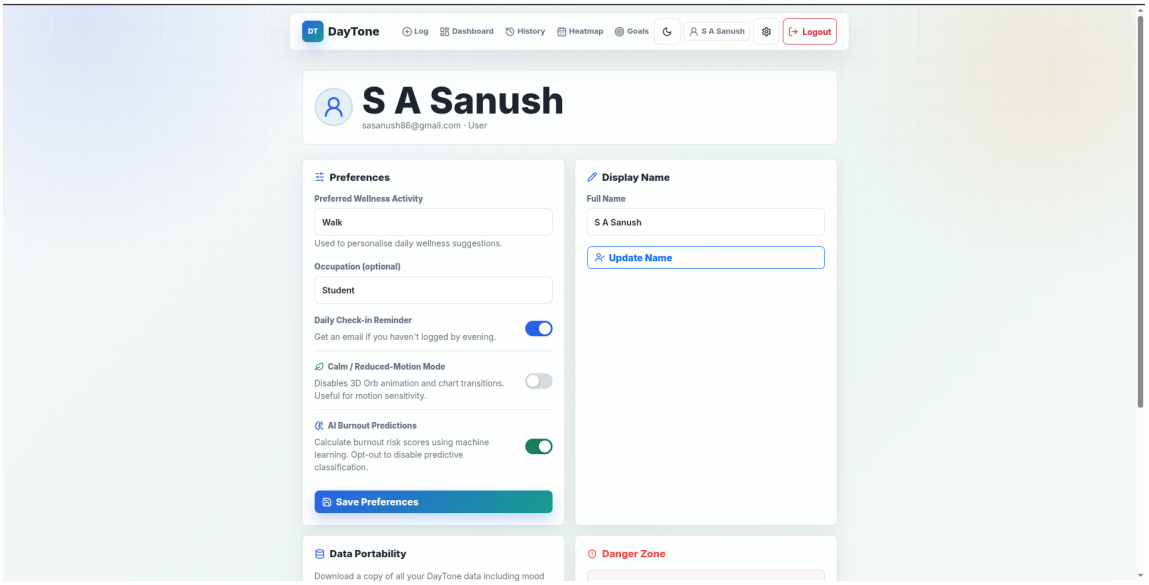


Figure 10: Profile and preference settings for the logged-in user

9.1 SCREENSHOTS (CONTINUED)

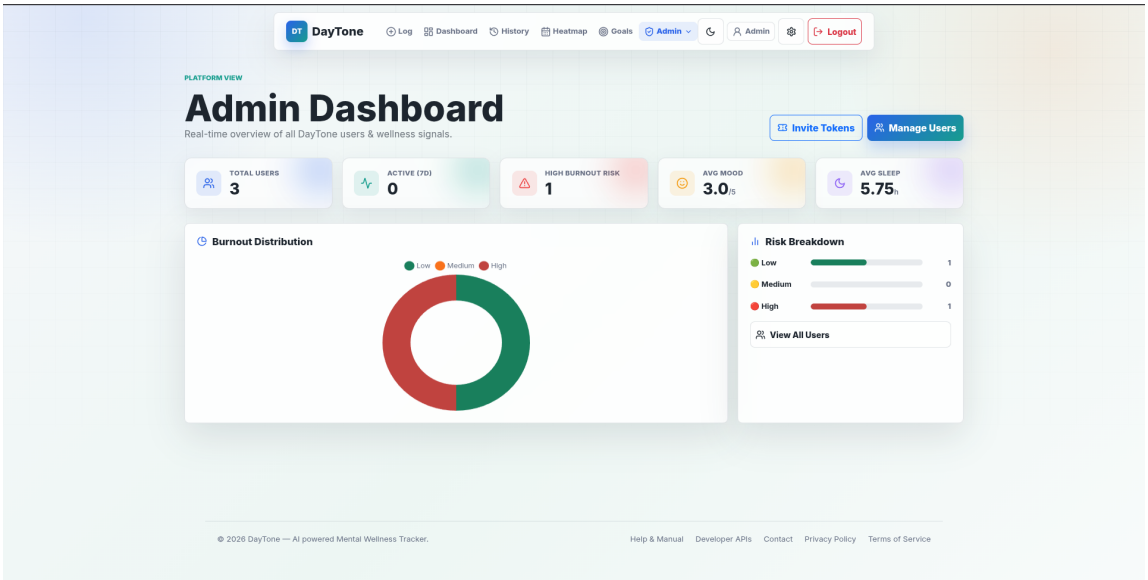


Figure 11: Admin dashboard with platform analytics and risk distribution

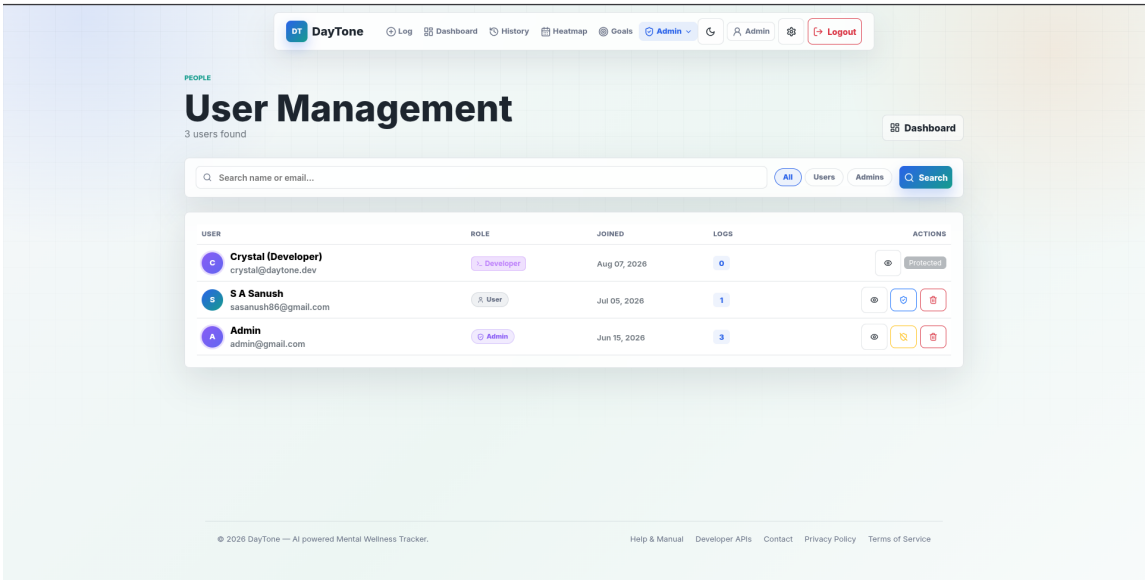


Figure 12: User management page with role and account controls

9.1 SCREENSHOTS (CONTINUED)

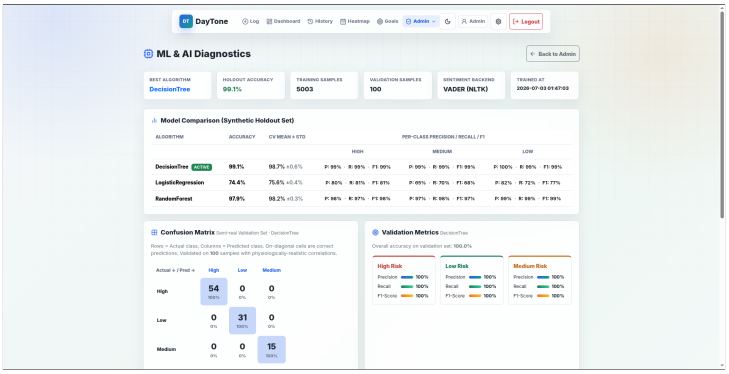


Figure 13: ML and AI diagnostics page with model comparison and metrics

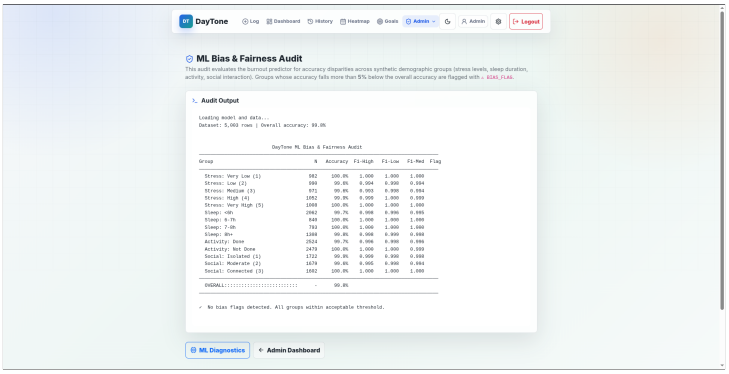


Figure 14: Bias and fairness audit output for the prediction model

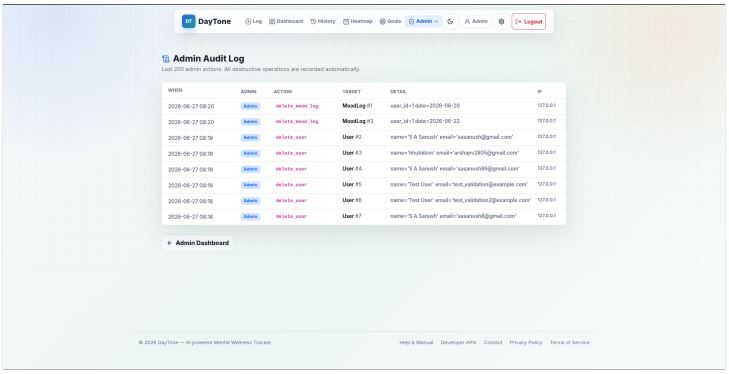


Figure 15: Admin audit log showing recorded administrative actions

9.1 SCREENSHOTS (CONTINUED)

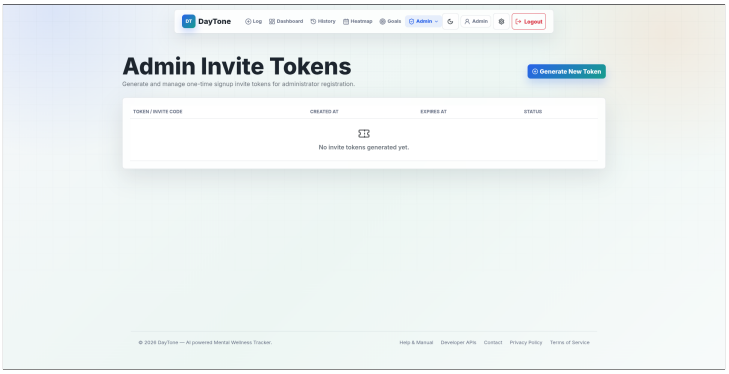


Figure 16: Admin invite token management screen

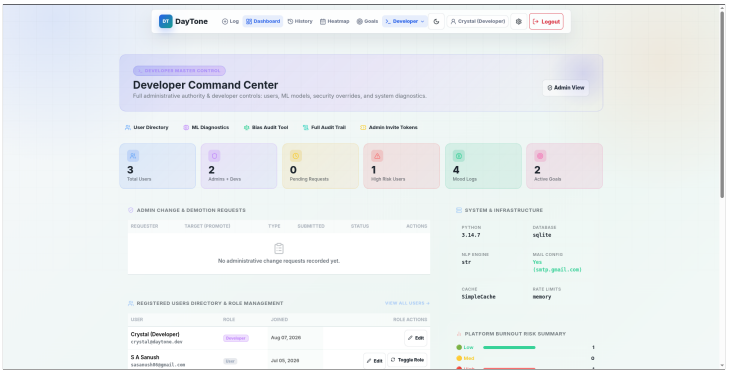


Figure 17: Developer command center with protected system controls

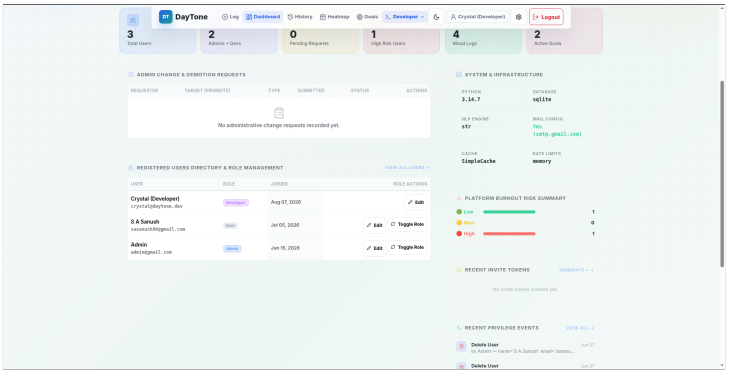


Figure 18: Developer dashboard with infrastructure and role management sections

CHAPTER 10
CONCLUSION

10.1 CONCLUSION

10.1 Conclusion is an important part of the DayTone project report. This section explains how the selected concept supports the overall objective of creating a secure, privacy-aware, AI-assisted mental wellness tracker. DayTone is designed with emphasis on privacy, usability, maintainability, and academic clarity. The implementation uses modular Flask blueprints, SQLAlchemy models, local NLP processing, and Scikit-learn prediction so that the system can be explained, tested, and improved in a structured way. The design also considers practical deployment requirements such as migrations, secure configuration, logging, data export, and administrator monitoring.

In this part of the project, the design decisions are connected to practical implementation. The section describes the role of the module, its interaction with other modules, the data it uses, and the expected result produced for users or administrators.

The same design philosophy is followed throughout DayTone: keep sensitive data protected, keep the interface simple, keep prediction results explainable, and keep the codebase maintainable for future enhancement.

In DayTone, 10.1 Conclusion is treated as a practical part of the complete wellness tracking workflow. The project does not present this topic only as a theoretical concept it connects the topic with actual screens, database records, backend routes, model outputs, and user decisions. This makes the report useful for explaining how the application works internally and how each module contributes to the final user experience.

The implementation follows a privacy-aware design approach. User wellness data may include personal habits, emotional reflections, stress values, and sleep patterns, so every major module is planned with confidentiality and controlled access in mind. Authentication, role-based pages, optional encryption, CSRF protection, and audit records support this requirement throughout the application.

From a software engineering perspective, the topic is implemented using modular Flask blueprints and reusable helper utilities. This makes the project easier to test and maintain because authentication, mood logging, administration, prediction, reports, mail, suggestions, and analytics are kept in separate areas of the codebase. A modular structure also makes future enhancement possible without rewriting the entire system.

From the user perspective, the topic is important because DayTone must remain simple and understandable. The system converts technical analysis into visible outputs such as dashboard cards, charts, heatmaps, risk labels, suggestions, history tables, and downloadable reports. The user is not expected to understand machine learning internals in order to benefit from the result.

In DayTone, 10.1 Conclusion is treated as a practical part of the complete wellness tracking workflow. The project does not present this topic only as a theoretical concept it connects the topic with actual screens, database records, backend routes, model outputs, and user decisions. This makes the report useful for explaining how the application works internally and how each module contributes to the final user experience.

10.2 FUTURE ENHANCEMENT

Future versions can improve prediction reliability by collecting consent-based real usage data and periodically retraining the Decision Tree model or comparing it again with ensemble models such as Random Forest. A larger dataset can help reduce bias and make predictions more representative of different student and professional routines.

The system can also add counsellor-facing workflows, anonymous institutional analytics, mobile push notifications, multilingual support, wearable device integration, and stronger explainable AI summaries. These additions would make DayTone more useful while still preserving privacy and user control.

Another enhancement is to introduce adaptive suggestions. Instead of showing only rule-based tips, the system can learn which suggestions users find useful and recommend activities based on previous improvement patterns.

10.2 Future Enhancement is an important part of the DayTone project report. This section explains how the selected concept supports the overall objective of creating a secure, privacy-aware, AI-assisted mental wellness tracker. DayTone is designed with emphasis on privacy, usability, maintainability, and academic clarity. The implementation uses modular Flask blueprints, SQLAlchemy models, local NLP processing, and Scikit-learn prediction so that the system can be explained, tested, and improved in a structured way. The design also considers practical deployment requirements such as migrations, secure configuration, logging, data export, and administrator monitoring.

In this part of the project, the design decisions are connected to practical implementation. The section describes the role of the module, its interaction with other modules, the data it uses, and the expected result produced for users or administrators.

The same design philosophy is followed throughout DayTone: keep sensitive data protected, keep the interface simple, keep prediction results explainable, and keep the codebase maintainable for future enhancement.

In DayTone, 10.2 Future Enhancement is treated as a practical part of the complete wellness tracking workflow. The project does not present this topic only as a theoretical concept it connects the topic with actual screens, database records, backend routes, model outputs, and user decisions. This makes the report useful for explaining how the application works internally and how each module contributes to the final user experience.

The implementation follows a privacy-aware design approach. User wellness data may include personal habits, emotional reflections, stress values, and sleep patterns, so every major module is planned with confidentiality and controlled access in mind. Authentication, role-based pages, optional encryption, CSRF protection, and audit records support this requirement throughout the application.

REFERENCES

1. Grinberg, M., Flask Web Development, O Reilly Media.

2. Pallets Projects, Flask Documentation.

3. SQLAlchemy Project, SQLAlchemy ORM Documentation.

4. Scikit-learn Developers, Decision Tree Classifier Documentation.

5. Hutto, C. J. and Gilbert, E., VADER: A Parsimonious Rule-based Model for Sentiment Analysis.

6. ReportLab User Guide, PDF Generation in Python.

7. OWASP Foundation, Web Application Security Guidelines.

8. Bootstrap Documentation, Responsive Frontend Components.

9. Chart.js Documentation, Browser-based Data Visualization.

10. DayTone source code, project documentation, model card, and testing files.

References is an important part of the DayTone project report. This section explains how the selected concept supports the overall objective of creating a secure, privacy-aware, AI-assisted mental wellness tracker. DayTone is designed with emphasis on privacy, usability, maintainability, and academic clarity. The implementation uses modular Flask blueprints, SQLAlchemy models, local NLP processing, and Scikit-learn prediction so that the system can be explained, tested, and improved in a structured way. The design also considers practical deployment requirements such as migrations, secure configuration, logging, data export, and administrator monitoring.

In this part of the project, the design decisions are connected to practical implementation. The section describes the role of the module, its interaction with other modules, the data it uses, and the expected result produced for users or administrators.

The same design philosophy is followed throughout DayTone: keep sensitive data protected, keep the interface simple, keep prediction results explainable, and keep the codebase maintainable for future enhancement.

In DayTone, References is treated as a practical part of the complete wellness tracking workflow. The project does not present this topic only as a theoretical concept it connects the topic with actual screens, database records, backend routes, model outputs, and user decisions. This makes the report useful for explaining how the application works internally and how each module contributes to the final user experience.